

[Day 2 종합실습] 실행결과 및 코드 분석 의견

SKALA 데이터분석 및 AIOps | End-to-End 데이터 분석 파이프라인 | 광주캠퍼스 4반 박영서 | 2026-07-21

실행결과 요약

- 실행환경: Python 3.11.15 / venv + requirements.txt로 패키지 관리 (pandas, polars, pyarrow, scikit-learn, scipy, seaborn, plotly, joblib, pytest, ruff)
- UCI Adult Census Income **32,561행 × 15열**을 Pandas와 Polars 두 방식으로 로딩해 shape·결측치 총계가 완전히 일치함을 확인 (양쪽 4,262건)
- 로딩 속도는 예열 2회 후 10회 측정 중앙값 기준 **Pandas 0.0155초 / Polars 0.0060초로 Polars가 2.58배 빨랐음** (1회 측정은 결과가 뒤집혀 신뢰할 수 없었음 — 아래 의견 참조)
- 중복 24행 제거, 범주형 결측 4,261건을 최빈값으로 대체해 32,537행 확보. 수치형 결측은 0건
- Seaborn 2×2 정적 차트와 Plotly 인터랙티브 차트 생성 — 전 그래프에 한글 제목·축 레이블 지정
- t-test(hours-per-week × income)와 카이제곱 독립성 검정(sex × income) 수행. 두 검정 모두 $p < 1e-308$ 로 귀무가설 기각하고, **해석 문장을 리포트에 자동 기록**
- ColumnTransformer + RandomForest를 하나의 sklearn Pipeline 객체로 구성. Accuracy 0.8175 / F1(macro) 0.7842 / **ROC-AUC 0.9213**
- joblib으로 모델을 저장한 뒤 다시 불러와 5건 예측까지 수행해 재현성 검증 성공
- 수집부터 리포트까지 6단계 전 과정이 **1.80초**(로딩 벤치마크 24회 포함)에 끝나고 report.md 167줄이 자동 생성됨
- pytest 9건 전부 통과, ruff check · ruff format 모두 통과

측정 결과

1) 로딩 성능 — Pandas vs Polars (동일 원본 3.8MB, 32,561행)

측정 방식	Pandas	Polars	결론
1회 측정 (첫 실행)	0.022s	0.044s	Pandas 2.04배 빠름
1회 측정 (재실행)	0.018s	0.010s	Polars 1.84배 빠름 (뒤집힘)
예열 2회 후 10회 중앙값	0.0155s	0.0060s	Polars 2.58배 빠름

shape(32561, 15)와 결측치 총계(4,262건)는 세 측정 모두 양쪽이 완전히 일치. 반복 측정 시 Polars 편차는 5.9~6.2ms로 안정적

2) 통계 검정 — p-value와 효과크기

검정	대상	통계량	p-value	효과크기
Welch t-test	hours-per-week × income	$t = -45.095$	$< 1e-308$	Cohen's $d = 0.552$
카이제곱 독립성	sex × income	$\chi^2 = 1516.54$ (df=1)	$< 1e-308$	Cramér's $V = 0.216$

평균 차이는 주당 6.63시간(38.84 → 45.47). 효과크기는 별도 계산해 덧붙인 값임

3) 모델 성능 — RandomForest Pipeline (학습 26,029행 / 테스트 6,508행)

지표	값	클래스	precision	recall	f1
Accuracy	0.8175	<=50K	0.955	0.797	0.869
F1 (macro)	0.7842	>50K	0.580	0.882	0.700
ROC-AUC	0.9213	다수 클래스 예측 기준선 = 75.9%			

분석 결과에 대한 본인 의견

한 번만 측정한 값으로 결론을 냈다가 뒤집혔다

처음 파이프라인을 돌렸을 때 Pandas 0.022초, Polars 0.044초가 나왔습니다. "데이터가 작아서 Polars의 초기화 비용이 이득을 넘었다"는 그럴듯한 해석을 붙여 리포트에 적었는데, **다시 실행하니 Polars가 1.84배 빠르다고 나오면서 결론이 뒤집혔습니다**. 해석이 틀린 게 아니라 측정이 틀린 것이었습니다.

예열 2회를 버리고 10회를 측정해 중앙값을 내자 **Polars 6.0ms, Pandas 15.6ms로 Polars가 2.6배 빨랐고**, 3회 연속 실행해도 값이 거의 흔들리지 않았습니다(Polars 5.9~6.0ms). 1회 측정에는 Polars의 스레드 풀 초기화 비용이 통째로 섞여 있었던 것입니다.

문제는 **Day 1에서 이미 같은 실수를 했다는 점**입니다. 그때도 Parquet 읽기가 184ms로 나와 워밍업을 넣어 1.0ms로 잡았고, "측정값이 상식과 어긋날 때 결론부터 쓰면 틀린 해석이 남는다"고 적었습니다. 이번엔 값이 상식과 어긋나 보이지 않아서 — 오히려 "작은 데이터에선 Polars가 느리다"는 그럴듯한 설명이 붙어서 — 의심하지 않고 넘어갔습니다. **이상한 값보다 위험한 건 그럴듯한 값이라는 걸 배웠습니다**. 그래서 측정 로직 자체를 `benchmark_loaders()` 로 분리해 예열·반복·중앙값을 강제하도록 고쳤습니다.

p-value가 같아도 결론의 무게는 달랐다

t-test와 카이제곱 모두 p가 1e-308 미만으로 나와, 숫자만 보면 두 결과가 똑같이 강력해 보입니다. 그런데 **표본이 32,537건이면 아주 작은 차이도 유의하게 나옵니다**. 실제로 5%만 뽑아 다시 검정해도 p는 1.75e-18로 여전히 유의했습니다. 그래서 효과크기를 따로 계산해봤더니 근무시간 차이는 Cohen's d 0.552로 중간 수준이었지만, 성별과 소득의 연관은 **Cramér's V 0.216으로 약한 편**이었습니다. "유의하다"와 "차이가 크다"는 다른 말이고, p-value만 보고 결론을 쓰면 과장이 된다는 걸 확인했습니다.

정확도가 낮아진 것은 의도한 결과였다

소득 분포가 76:24로 불균형이라 `class_weight="balanced"` 를 줬습니다. 그 결과 정확도는 0.8175로, 이 옵션 없이 학습했을 때보다 낮습니다. 하지만 목적이 "고소득자를 찾아내는 것"이라면 정확도보다 **>50K**의 재현율이 중요하고, 이 값이 **0.882**로 올라갔습니다. 대신 정밀도가 0.580으로 떨어져, 고소득으로 예측한 사람 10명 중 4명은 실제로 아니라는 뜻입니다. 어느 쪽이 맞는지는 이 모델을 어디에 쓰느냐가 정해야 하는 문제라 판단했고, 정확도만 높은 모델이 좋은 모델은 아니라는 걸 수치로 확인했습니다. ROC-AUC 0.9213은 임계값과 무관한 지표라 이 트레이드오프에 영향받지 않습니다.

결측치 대체를 분할 전에 해서 미세한 누수를 남겼다

파이프라인이 `clean_data()` 로 전체 데이터의 최빈값을 구해 결측치를 채운 뒤(80행), 그 다음에 `train_test_split` 을 합니다(107행). 즉 **대체값을 구할 때 테스트셋이 섞여 들어갔습니다**. 32,000건에서 범주형 최빈값은 거의 흔들리지 않아 성능에 미치는 영향은 사실상 없지만, 원칙적으로는 `SimpleImputer` 를 `ColumnTransformer` 안에 넣어 학습 폴드에서만 대체값을 학습하게 하는 것이 맞습니다. 인코딩과 스케일링은 Pipeline 안에 제대로 넣었으면서 결측 대체만 밖으로 나와 있었고, 이걸 지금 구조에서 남은 명확한 결함입니다.

`fnlwg`는 숫자라고 해서 피처가 되지는 않았다

수치형 컬럼이라 처음엔 그냥 학습에 넣었는데, `fnlwgt` 는 인구조사 **표본 가중치**입니다. "이 응답자가 전체 인구 몇 명을 대표하는가"를 뜻하는 조사 설계상의 값이라 개인의 소득과 인과관계가 없습니다. 상관계수도 전 변수에 대해 $-0.076 \sim 0.000$ 으로 사실상 무관했습니다. 그대로 두면 RandomForest가 값이 다양한 이 컬럼에서 의미 없는 분기를 만들 수 있어 피처에서 제외하고 그 이유를 `config.py` 에 주석으로 남겼습니다. 컬럼의 자료형이 아니라 그 값이 무엇을 의미하는지를 먼저 봐야 한다는 점을 배웠습니다.

종합 의견 — 이 선택들을 왜 했는가

제시된 데이터셋 중 Adult Census Income을 고른 이유

이번 실습은 수집·전처리·시각화·통계 검정·머신러닝을 한 **파이프라인 안에서 전부 다뤄야** 했습니다. 그래서 어느 한 단계라도 "쓸 일이 없어" 형식적으로 넘어가게 되는 데이터는 피하려고 했습니다. Adult는 그 조건을 모두 갖고 있었습니다.

- **결측치가 진짜로 있다** — `workclass` · `occupation` · `native-country` 3개 컬럼에 ? 로 들어 있어, 결측 처리 전략을 실제로 고민해야 합니다
- **수치형과 범주형이 섞여 있다** — 수치형 5개, 범주형 7개(원-핫 시 83차원)라 `ColumnTransformer` 로 서로 다른 전처리를 묶는 구조가 자연스럽게 필요해집니다
- **클래스가 불균형하다** — 76:24라 정확도만 보면 안 되는 상황이 만들어져, 지표 선택을 설명해야 합니다
- **통계 검정을 붙일 자리가 있다** — 연속형(`hours-per-week`)과 범주형(`sex`)이 모두 타깃과 연결돼 t-test와 카이제곱을 각각 쓸 수 있습니다

즉 "분석하기 좋은 깨끗한 데이터"가 아니라 **배워야 할 처리를 전부 요구하는 데이터**여서 선택했습니다.

전처리를 이렇게 한 이유

결측치는 삭제하지 않고 대체했습니다. 결측이 있는 3개 컬럼의 비율은 각각 5.64%, 5.66%, 1.79%입니다. 개별로는 작아 보이지만 결측이 있는 행을 모두 지우면 손실이 누적됩니다. 실제로 같은 데이터를 행 삭제 방식으로 처리하면 30,139행이 남아 **2,422 행(7.4%)이 사라집니다.** 대체 방식은 32,537행을 지켰습니다. 범주형은 최빈값, 수치형은 (이상치에 덜 흔들리는) 중앙값을 쓰기로 했고, 결과적으로 수치형 결측은 0건이라 실제로는 범주형 4,261건만 대체됐습니다.

그런데 대체한 뒤에 분포가 얼마나 틀어졌는지는 확인하지 않았습다. 뒤늦게 재보니 컬럼마다 결과가 전혀 달랐습다.

컬럼	최빈값	대체 건수	대체 전	대체 후	증가폭
workclass	Private	1,836	73.87%	75.33%	+1.46%p
occupation	Prof-specialty	1,843	13.48%	18.38%	+4.90%p
native-country	United-States	583	91.22%	91.39%	+0.17%p

`native-country` 는 이미 91%가 미국이라 583건을 채워도 0.17%p밖에 안 움직입니다. 반면 **occupation 은 최빈값이 13.48%에 불과한 평평한 분포**(14개 값 중 상위 3개가 13.48/13.34/13.24%로 거의 동률)여서, 1,843건을 한 값에 몰아주자 그 범주가 18.38%로 **+4.90%p 부풀려졌습니다.** 원래 1위와 2위의 차이가 0.14%p였던 것을 생각하면 순위 구조 자체를 흔든 셈입니다.

"결측 비율이 6% 미만이니 대체해도 안전하다"고 판단했는데, **봐야 했던 것은 결측 비율이 아니라 최빈값이 얼마나 지배적인가**였습니다. 지배적 범주가 있는 컬럼에는 최빈값 대체가 무해하지만, 평평한 분포에는 없던 편향을 만듭니다. `occupation` 은 `Unknown` 이라는 별도 범주로 두는 편이 나았을 것 같습니다 — 응답하지 않았다는 사실 자체가 정보일 수 있기 때문입니다. 이 진단을 리포트에 자동 출력되도록 넣어, 다음에는 대체 전에 판단할 수 있게 했습니다.

중복 24행은 제거했습니다. 설문 데이터에서 15개 컬럼이 전부 같은 행은 같은 사람이 두 번 집계됐을 가능성이 높다고 봤습니다. 다만 24행은 전체의 0.07%라 결과에 미치는 영향은 거의 없습니다.

피처는 자료형이 아니라 의미로 걸렸습니다. `fnlwgt` 는 수치형이지만 인구조사 표본 가중치라 제외했고, `education` 은 `education-num` 과 1:1 대응이라 수치형 하나만 남겼습니다. 컬럼이 숫자로 되어 있다는 사실만으로 피처가 되지는 않는다고 판단했습니다.

같은 데이터, 다른 선택 — 조 안의 두 구현을 비교하며

같은 조에서 동일한 데이터로 만든 다른 구현과 비교해 보니, 핵심 지점마다 반대되는 선택을 했고 그만큼 결과가 갈렸습니다.

항목	이번 구현	조 내 다른 구현
결측 처리	최빈값 대체 (32,537행 유지)	해당 행 삭제 (30,139행, 2,422행 손실)
모델	RandomForest + <code>class_weight="balanced"</code>	LogisticRegression
Accuracy	0.8175	0.8461
ROC-AUC	0.9213	0.9006
>50K 재현율	0.882	0.6009

정확도만 보면 이번 구현이 졌습니다. 하지만 >50K 재현율은 0.882 대 0.601로 차이가 큼니다. 불균형 보정을 하지 않으면 모델이 다수 클래스로 기울어 정확도는 올라가지만 소수 클래스를 놓친다는 것이, 두 결과를 나란히 놓고 보니 분명해졌습니다. 반대로 ROC-AUC는 0.9213 대 0.9006으로 큰 차이가 없어, **두 모델의 판별력 자체는 비슷하고 갈린 것은 임계값 근처의 운영 방침이었다고 정리할 수 있었습니다.**

결측 처리도 마찬가지입니다. 행을 지우면 분포는 왜곡되지 않지만 표본 7.4%를 잃고, 대체하면 표본은 지키지만 위에서 본 것처럼 특정 컬럼의 분포가 틀어집니다. **어느 쪽도 공짜가 아니었고, 무엇을 지불할지 고르는 문제였습니다.**

혼자 만들었을 때는 "정확도 0.8175"라는 숫자가 잘한 것인지 못한 것인지 판단할 기준이 없었습니다. 같은 데이터로 다른 선택을 한 결과가 옆에 있어야 **내 선택이 무엇을 얻고 무엇을 포기한 것인지** 말할 수 있다는 점이 이번 팀 실습에서 가장 크게 남았습니다.

추가 의견 (개선 사항, 코드 품질 개선 측면)

- 결측 대체를 `SimpleImputer` 로 `ColumnTransformer` 안으로 옮겨, 위에 적은 데이터 누수를 구조적으로 없애야 합니다
- 현재는 홀드아웃 1회 평가라 `cross_val_score` 로 폴드별 편차를 확인해야 성능 수치를 신뢰할 수 있습니다. 하이퍼파라미터도 기본값이라 `GridSearchCV` 탐색 여지가 남아 있습니다
- `class_weight` 대신 예측 확률의 **임계값을 직접 조정**하면 정밀도-재현율 균형을 용도에 맞게 세밀하게 맞출 수 있습니다. SMOTE와의 비교도 해보고 싶습니다
- `capital-gain` 은 75%가 0인데 최댓값이 99,999로 극단적으로 편향돼 있습니다. 로그 변환이나 구간화로 완화하면 트리 분기가 안정될 것 같습니다
- 성별과 소득이 통계적으로 연관 있는 데이터로 학습했으므로, 모델이 이 편향을 그대로 재생산할 수 있습니다. 그룹별 재현율 격차 같은 **공정성 지표**를 평가에 포함해야 합니다
- 테스트가 9건이라 통계·모델 함수의 정상 경로 위주입니다. `pytest --cov` 로 커버리지를 수치화해 검증이 닿지 않는 분기를 관리하면 좋겠습니다

- 데이터가 수백만 행으로 커지면 전처리 전체를 Polars Lazy API로 이관할 여지가 있습니다. 다만 이번처럼 **측정 방법을 먼저 고정해 놓고** 비교해야 의미 있는 판단이 됩니다
- 지금은 로딩만 벤치마크합니다. 전처리·학습 단계도 같은 방식(예열+반복+중양값)으로 재면 어느 구간이 실제 병목인지 근거를 갖고 말할 수 있습니다

실행결과 화면 캡처

아래 세 장은 macOS 터미널에서 직접 실행한 화면을 캡처한 것입니다.

1. python -m src.run_pipeline — 수집 → 로딩비교 → 전처리 → 시각화 → 통계 → 학습·저장

```
scala-gwangju-4class-team2 — 광주캠퍼스_4반_박영서_day2종합실습 — zsh — 209x36

(.venv) parkyoungseo@bag-yeongseoui-MacBookPro scala-gwangju-4class-team2 % python -m src.run_pipeline
2026-07-21 16:43:03,884 INFO | run_pipeline | =====
2026-07-21 16:43:03,885 INFO | run_pipeline | [1/6] 데이터 수집
2026-07-21 16:43:03,885 INFO | clean | 원본 캐시 사용: /Users/parkyoungseo/workspace/scala-gwangju-4class-team2/data/raw/adult.data (3881.2 KB)
2026-07-21 16:43:03,885 INFO | run_pipeline | [2/6] Pandas / Polars 로딩 및 비교
2026-07-21 16:43:04,222 INFO | clean | 로딩 벤치마크 | 예열 2회 후 10회 중앙값 — Pandas 0.0155s, Polars 0.0060s
2026-07-21 16:43:04,231 INFO | clean | 로딩 비교 | shape 일치=True, 연속 일치=True, Pandas 0.016s vs Polars 0.006s
2026-07-21 16:43:04,231 INFO | run_pipeline | [3/6] 기본 EDA 및 전처리
2026-07-21 16:43:04,247 INFO | clean | 기본 EDA | 32861행 15컬 + 중복 24행, 연속 컬럼 3개
2026-07-21 16:43:04,267 INFO | clean | 전처리 완료 | 중복 24행 제거, 범주형 4261건 수치형 0건 대체, 최종 32537행
2026-07-21 16:43:04,320 INFO | run_pipeline | [4/6] 시각화
2026-07-21 16:43:04,623 INFO | viz | Seaborn 정적 차트 저장: /Users/parkyoungseo/workspace/scala-gwangju-4class-team2/outputs/figures/eda_seaborn.png
2026-07-21 16:43:04,777 INFO | viz | Plotly 인터랙티브 차트 저장: /Users/parkyoungseo/workspace/scala-gwangju-4class-team2/outputs/figures/eda_plotly.html
2026-07-21 16:43:04,777 INFO | run_pipeline | [5/6] 통계 분석
2026-07-21 16:43:04,780 INFO | stats | 기술통계 산출 완료: 6개 수치형 컬럼
2026-07-21 16:43:04,781 INFO | stats | 상관행렬 산출 완료: 6x6
2026-07-21 16:43:04,784 INFO | stats | t-test | hours-per-week: <=50K(38.84) vs >50K(45.47), t=-45.095, p< 1e-308 < 0.05 → 귀무가설 기각. '<=50K' 그룹과 '>50K' 그룹의 hours-per-week 평균에 차이가 있다 (통계적으로 유의).
2026-07-21 16:43:04,787 INFO | stats | chi2 | sex x income: chi2=1516.540, dof=1, p< 1e-388 < 0.05 → 귀무가설 기각. 'sex'와 'income'는 서로 연관되어 있다 (통계적으로 유의).
2026-07-21 16:43:04,787 INFO | run_pipeline | [6/6] ML Pipeline 학습 및 저장
2026-07-21 16:43:04,796 INFO | model | 데이터 분할 | 학습 2602행, 테스트 6508행
2026-07-21 16:43:05,483 INFO | model | 모델 학습 완료: RandomForestClassifier
2026-07-21 16:43:05,614 INFO | model | 평가 지표 | accuracy=0.8175, f1(macro)=0.7842, ROC-AUC=0.9213
2026-07-21 16:43:05,641 INFO | model | 모델 저장: /Users/parkyoungseo/workspace/scala-gwangju-4class-team2/outputs/models/income_pipeline.joblib (26855.7 KB)
2026-07-21 16:43:05,684 INFO | model | 모델 재로딩 검증 성공 | 생성 5건 예측 완료
2026-07-21 16:43:05,687 INFO | report | 리포트 생성 완료: /Users/parkyoungseo/workspace/scala-gwangju-4class-team2/outputs/report.md (167줄)
2026-07-21 16:43:05,687 INFO | run_pipeline | =====
2026-07-21 16:43:05,687 INFO | run_pipeline | 파이프라인 완료 (1.80초) → /Users/parkyoungseo/workspace/scala-gwangju-4class-team2/outputs/report.md
(.venv) parkyoungseo@bag-yeongseoui-MacBookPro scala-gwangju-4class-team2 %
```

로딩 벤치마크가 예열 2회 후 10회 중앙값을 산출(Pandas 0.0155s / Polars 0.0060s). 개별 로딩 로그는 DEBUG로 낮춰 콘솔에는 요약만 남김. shape-결측치가 양쪽 일치하고, 두 통계 검정의 해석 문장이 로그에 그대로 출력됨. 모델 저장 후 재로딩 검증까지 성공하고 report.md 167줄이 자동 생성됨. (경로는 지면 관계상 프로젝트 상대경로로 줄여 표기)

2. pytest -v

```
scala-gwangju-4class-team2 — 광주캠퍼스_4반_박영서_day2종합실습 — zsh — 180x28

(.venv) parkyoungseo@bag-yeongseoui-MacBookPro scala-gwangju-4class-team2 % pytest -v
===== test session starts =====
platform darwin — Python 3.11.15, pytest-8.3.4, pluggy-1.6.0 — /Users/parkyoungseo/workspace/scala-gwangju-4class-team2/.venv/bin/python3.11
cachedir: .pytest_cache
rootdir: /Users/parkyoungseo/workspace/scala-gwangju-4class-team2
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.2
collected 9 items

tests/test_pipeline.py::test_clean_data_removes_duplicates_and_nulls PASSED [ 11%]
tests/test_pipeline.py::test_clean_data_does_not_mutate_input PASSED [ 22%]
tests/test_pipeline.py::test_basic_eda_keys PASSED [ 33%]
tests/test_pipeline.py::test_run_ttest_returns_interpretation PASSED [ 44%]
tests/test_pipeline.py::test_run_chi2_raises_on_missing_column PASSED [ 55%]
tests/test_pipeline.py::test_run_chi2_contingency_shape PASSED [ 66%]
tests/test_pipeline.py::test_pipeline_fit_predict PASSED [ 77%]
tests/test_pipeline.py::test_train_and_evaluate_raises_without_target PASSED [ 88%]
tests/test_pipeline.py::test_save_and_reload_model PASSED [100%]

===== 9 passed in 1.15s =====
(.venv) parkyoungseo@bag-yeongseoui-MacBookPro scala-gwangju-4class-team2 %
```

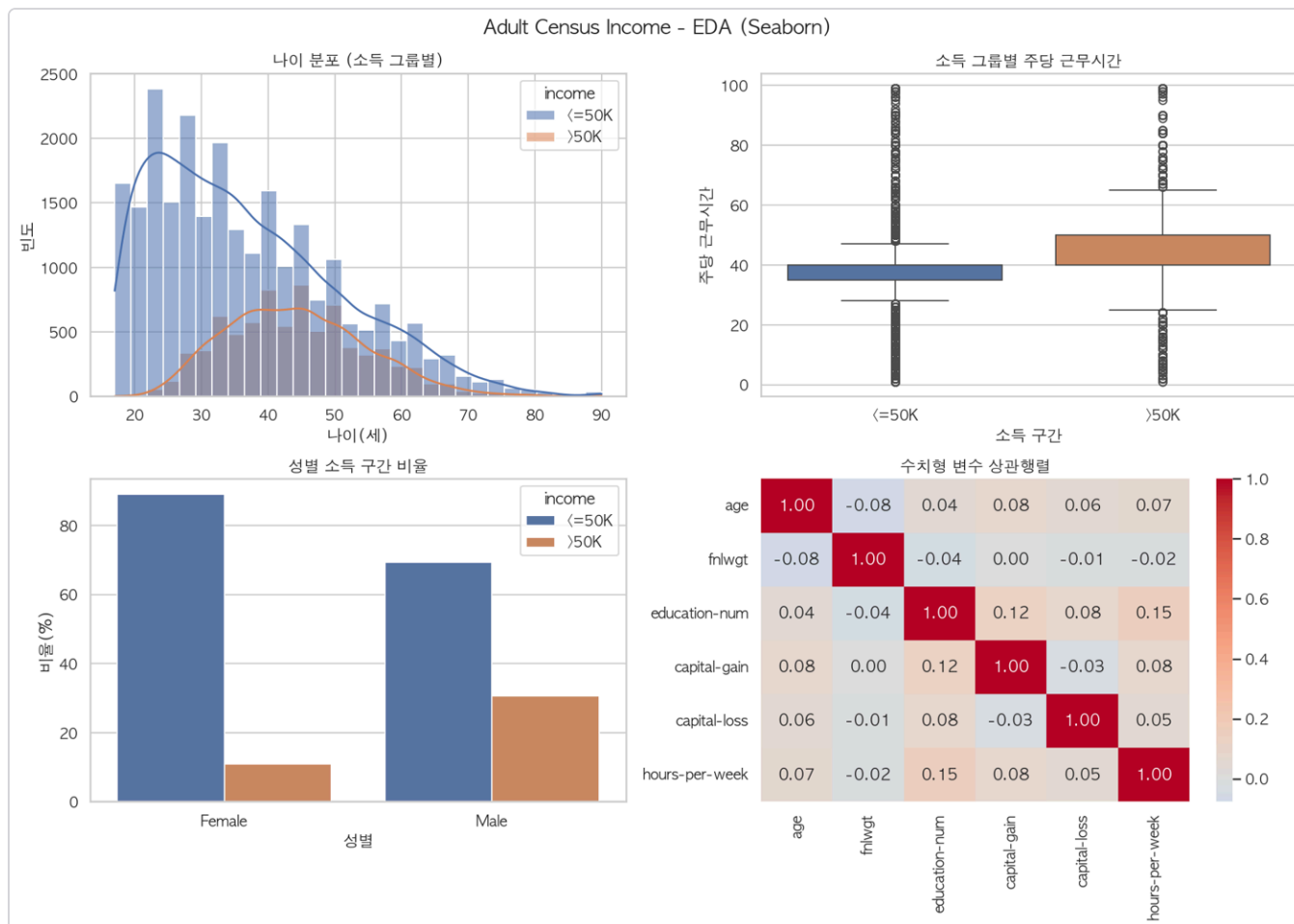
전처리 4건 + 통계 검정 3건 + 모델 학습·저장 2건, 총 9건 전부 PASSED. 입력 DataFrame을 훼손하지 않는지, 타깃 컬럼이 없을 때 예외를 내는지도 검증함

3. ruff check . / ruff format --check .

```
scala-gwangju-4class-team2 — 광주캠퍼스_4반_박영서_day2종합실습 — zsh — 166x10

(.venv) parkyoungseo@bag-yeongseoui-MacBookPro scala-gwangju-4class-team2 % ruff check . && ruff format --check .
All checks passed!
10 files already formatted
(.venv) parkyoungseo@bag-yeongseoui-MacBookPro scala-gwangju-4class-team2 %
```

4. 생성된 시각화 산출물 — outputs/figures/eda_seaborn.png



파이프라인 [4/6] 단계에서 자동 생성된 Seaborn 2x2 차트. 나이 분포(소득 그룹별 히스토그램+KDE), 소득 그룹별 주당 근무시간(박스플롯), 성별 소득 구간 비율(막대), 수치형 변수 상관행렬(히트맵). 네 그래프 모두 제목과 축 레이블을 한글로 지정함. 이와 별도로 Plotly 인터랙티브 차트(eda_plotly.html)도 생성됨

채점 기준 대응

항목	대응 내용
데이터 준비 + 시각화 (35점)	Pandas·Polars 양쪽으로 로딩해 shape·결측치 일치 확인 및 속도 비교, 중복 24행 제거와 범주형 결측 4,261건 최빈값 대체, 기본 EDA 출력. Seaborn 정적 차트와 Plotly 인터랙티브 차트를 각각 생성하고 전 그래프에 제목·축 레이블 지정
통계 분석 + 머신러닝 (45점)	기술통계·상관계수 산출, t-test와 카이제곱 검정을 수행해 p-value 해석 문장 을 리포트에 자동 기록. ColumnTransformer + RandomForest를 하나의 Pipeline 객체로 구성해 Accuracy·F1·ROC-AUC 출력, joblib 저장 후 재로딩 검증까지 완료
자동화 + 발표 (20점)	<code>python -m src.run_pipeline</code> 한 번으로 6단계 전 과정이 실행되고 report.md가 자동 생성됨. 단계별 로그를 콘솔과 파일에 동시 기록
완성도 (10점)	8개 모듈 전부에 프로그램 개요·모듈 구성·변경 내역을 담은 머리말 docstring 작성, 함수 단위 docstring과 판단 근거 주석 기재. pytest 9건·ruff 통과, Git 커밋 이력 존재